Tag, You're Lagged!

Design and Optimization of a Multiplayer Network Application

Author: MEng. Thanh Hoang Le Hai — thanhhoang@hcmut.edu.vn

> **It's Not About the Game, It's About the Network!**
>
> A real-time multiplayer game is your testbed for designing an application-layer communication protocol. Your task is to make the game playable from localhost to LAN and the public Internet, preserve the game rules, and improve its networking behavior with measurable evidence.

> **Important: Do not redistribute assignment materials**
>
> **Do not re-upload, publish or redistribute this handout, the supplied source code or other provided assignment materials, in whole or in part, without prior written permission from the lecturer or relevant rights holder.** This includes public code repositories, document-sharing sites, social media and other public platforms. Use these materials only for authorized course activities and comply with the University's applicable regulations on digital learning materials and intellectual property. **During the course, do not publicly publish your assignment source code, solutions or submission materials, including your own work, to prevent copying.** Keep them private and submit through the designated course channels. After the course has ended, you may publish the work you independently developed, provided that you exclude this handout, supplied source code and other restricted course materials unless permission has been granted. The required public game deployment during the course permits access to the playable application; it does not authorize publication of its source-code package or assignment solutions.

1. Learning objectives

By completing this assignment, each team will demonstrate the ability to:

- **Analyze network application requirements:** timing, reliability, ordering, interaction latency, state freshness, and recovery.
- **Design an application-layer protocol:** message syntax, semantics, direction, timing, transport, and communication sequences.
- **Develop and deploy networked software:** implement a multiplayer client and server on localhost, LAN, and the Internet.
- **Improve communication:** choose and implement a networking optimization that addresses an identified limitation.
- **Evaluate designs experimentally:** preserve game correctness while measuring performance and explaining trade-offs.

2. Application requirements and workflow

Download the starter source ZIP from BK LMS and extract it locally. The starter contains a TypeScript browser client, an authoritative Python game server and fixed sample test scenarios. Extend it into a real-time multiplayer browser game that implements the rules in Section 3. You design its application-layer communication protocol, client/server state handling, synchronization strategy and user interface. **The emphasis is network application design and optimization, not visual complexity.**

Element	Assignment requirement
Primary task	Application-layer communication protocol design and networking optimization.
Client and server	TypeScript browser client and Python authoritative server, organized in one project.
Communication	Select and justify a protocol and transport; JSON over WebSocket is a possible starting point, not a mandatory solution.
Implementation path	BK LMS starter ZIP → Localhost → LAN → Public Internet → Optimization.
Deployment	Render Free (reference deployment) or a functionally equivalent Internet-accessible setup.
Performance evaluation	Fixed scenario input/output, network measurements and reproducible before/after comparisons.

What you will build

A browser-playable game with multiplayer rooms; an application-layer protocol designed and implemented by your team; a publicly accessible deployment; and experimental evidence demonstrating the behavior and trade-offs of at least one networking improvement.

Provided interface

The screenshot below shows the game interface included in the starter code provided for this assignment. Your team may modify the UI while preserving the game mechanics in Section 3. Each team should develop its own theme or concept to make the publicly deployed application recognizable and appealing to players outside the course. Choose a memorable app name, a coherent visual style and a clear presentation of the game's actions. The theme may change characters, item names and visual effects, while all mechanics, credit values, timing and match rules in Section 3 remain unchanged.

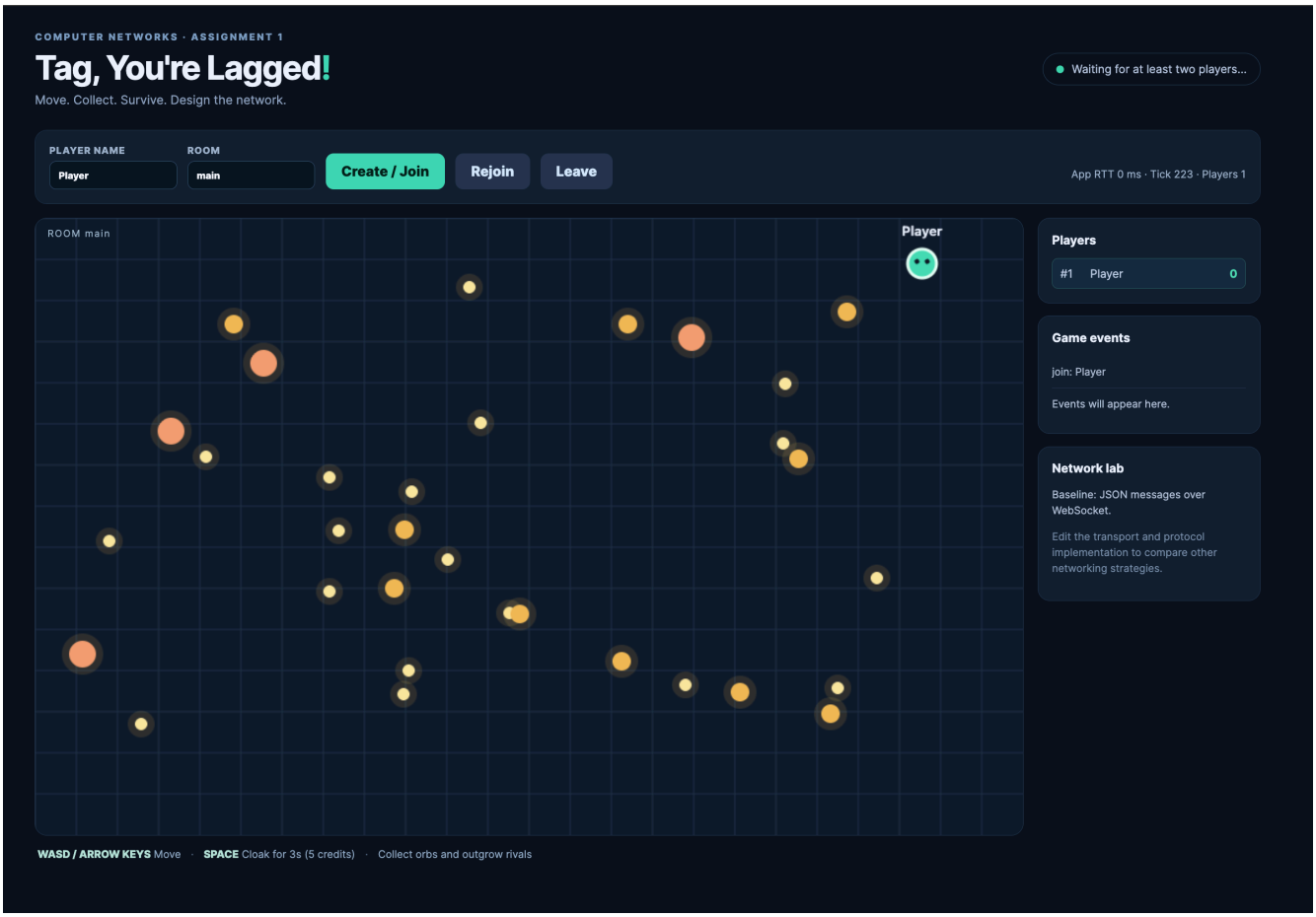


Figure 1: Provided game interface.

Example theme ideas

Starting from the provided skeleton, teams can create their own concepts. For example:

- **Vịt Chạy Deadline:** ducks compete for coffee and instant noodles while racing against deadlines.
- **Canteen Royale:** students compete for the last serving of food at the canteen.
- **Bug Hunter:** programmers collect bugs and sabotage their rivals' code.
- **Packet Panic:** packets escape routers and compete for bandwidth.

Express these ideas through characters, collectibles and visual effects that follow the fixed game rules; they do not introduce extra mechanics. Keep the correspondence clear in the interface and your report.

3. Fixed game mechanics

The following rules form the game's authoritative contract. Network protocol, serialization, state storage and synchronization may change; the game outcome must remain consistent with these rules.

Mechanic	Rule
Room and lifecycle	Create or join a game room (maximum 20 players). The match starts with at least two players, and new players may join during play. A round ends when one player remains alive.
Map and movement	A continuous 960 $\times$ 600 arena. Move in four cardinal directions (one at a time), using WASD or arrow keys. Authoritative game simulation runs at 20 ticks/s.
Collectibles	Small, medium and large items grant 1, 3 and 7 credits, respectively. Items respawn after collection. A collectible can be awarded only once per event.
Growth	Player radius increases with credits: $r(c) = 14 + \min(22,\ 2\sqrt{c})$.
Mobility trade-off	The greater a player's credit balance, the slower the player moves: $v(c) = \max(65,\ 170/(1 + 0.09\sqrt{c}))$ units/s.
Credit decay	Every 5 seconds of active play, a player loses $\lceil 0.02c \rceil$ credits (up to the available balance). The interval is measured per player from spawn or rejoin.
Invisibility	Spend 5 credits to become invisible to other players for 3 seconds (60 ticks). Cloaking does not prevent physical collision or elimination.
Player collision	When two players touch, the one with strictly more credits eliminates the other. If credits are equal, neither is eliminated.
Kill reward	The surviving attacker receives $\lceil 0.40c_{\text{victim}} \rceil$ credits from the defeated player.
Elimination and return	An eliminated player may rejoin with zero credits. After a finished match, a new participant or rejoin can start the next round.

Figure 2 compares player size and movement at 0, 25 and 100 credits on the same spatial scale. Arrows show the distance travelled in one second.

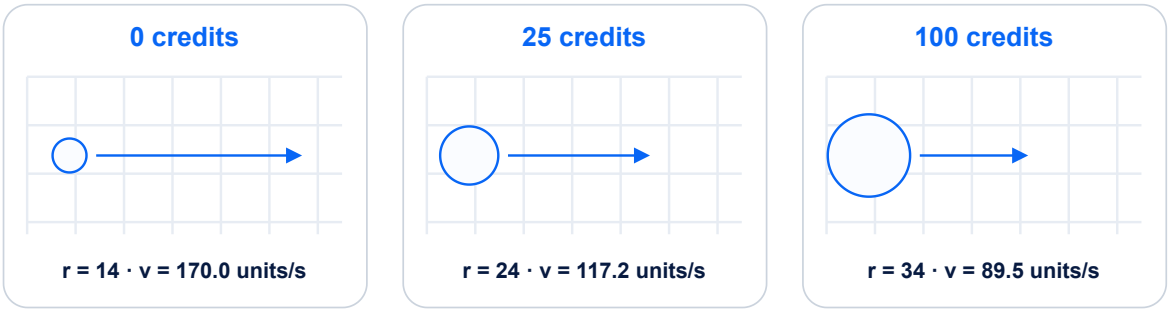


Figure 2: Player movement and size mechanisms.

Consistency and deterministic execution

- The server is authoritative for credit changes, collisions, elimination, cloaking and match outcomes. Clients transmit actions, not self-declared credits or positions.
- Tests use controlled starting conditions and predetermined action sequences. Each collectible is awarded to at most one player, even if several attempt to collect it simultaneously.
- Cloaked players are hidden from opponents' normal views, but remain part of authoritative collision and scoring decisions.
- Concurrent joins, action duplication, rejoin and disconnection must not violate the game rules.

> **Game outcomes are the correctness contract**
>
> Completed-match tests compare the **winner, round, final alive/eliminated roster, and remaining credits**. Exact agreement at each intermediate tick, snapshot, or player coordinate is **not required**.

4. Student-controlled networking design

Design an application-layer protocol that serves the game's interactions. A client–server implementation can satisfy the full assignment. Peer-assisted, P2P or hybrid designs are also permitted when demonstrated correctly.

Required protocol design work

- **Communication requirements:** classify actions, events and state updates according to latency, freshness, reliability and ordering requirements.
- **Message specification:** define each message's direction, fields, syntax, semantics, validation and handling rules.
- **Processing placement:** compare centralized server-side processing/routing with browser-side processing and direct networking. Justify where your design performs encoding, buffering, filtering, rendering and communication; explain the trade-offs in latency, traffic, server load, client workload and scalability. Authoritative game decisions remain on the Python game server.
- **Transport and exchange strategy:** justify your choice of WebSocket, HTTP short/long polling or another suitable approach; define update cadence, push/pull, batching, and recipients.
- **Synchronization:** explain state consistency, duplicates, stale messages, loss of connectivity, recovery, and client-visible state.
- **Design rationale:** document the trade-offs using protocol tables and message-sequence diagrams.

Example protocol table (reference only)

The following table illustrates the scope and level of detail to consider; it is not a complete protocol specification. Each team must create its own protocol tables, narrative descriptions and explanations based on its implemented design. Do not copy this sample table or its wording directly into the submission. Specify and justify your actual messages, fields, validation, exchange sequences and design decisions.

Message	Direction	Semantics	Design questions
`INPUT`	Client → Server	Direction and cloak action.	Frequency, ordering, stale input.
`STATE`	Server → Client	Player/item state or state changes.	Snapshot vs. delta, recipients.
`JOIN/REJOIN`	Client → Server	Enter room or new life.	Session identity and recovery.
`RESULT`	Server → Client	Finished round and credits.	Correctness and convergence.

Implementation boundaries

Area	Requirement
Fixed mechanics	Room lifecycle, map dimensions, movement, collectibles, growth, credit decay, invisibility, collisions and game results must follow Section 3.
Team-designed	Protocol messages, transport, server/client state representation, buffering, synchronization, reconnection and UI.
Extensible	Teams may create additional modules, tools, visual assets, test cases and communication strategies.

Browser-side networking and socket libraries are allowed. Teams must design their own application protocol, room behavior and synchronization rather than delegate these to a ready-made multiplayer or replication platform. UI and graphics libraries are also allowed.

Browser clients and player sessions

A **client** is a running browser application communicating with the game server. A tab is a client instance; the player session identifies its participant and must survive a reload. It is not a separate computer or a separate copy of the source code. For localhost multiplayer, run one game server and serve one client build; open the same game URL in two independent browser sessions on the same computer.

For two distinct players, use two different browsers (for example, Chrome and Firefox) or two separate browser profiles to obtain independent sessions. Two ordinary windows or tabs in the same browser profile usually share cookies and are not independent player sessions.

The provided starter code currently treats two tabs as two different players. Implement persistent player identity using a persistent cookie, `localStorage`, or an equivalent mechanism that shares identity across tabs in the same browser profile. Store a player/session identifier and associate it with server-side player information; browser-stored credits or positions must not override authoritative state. Reloading or opening another tab in the same session must not silently create a new player. Define how simultaneous tabs for one identity are handled, such as sharing the same identity or rejecting a duplicate active connection. Returning after elimination still follows the explicit rejoin rule: zero credits.

Figure 3 shows two browser sessions on one computer accessing the same game server.

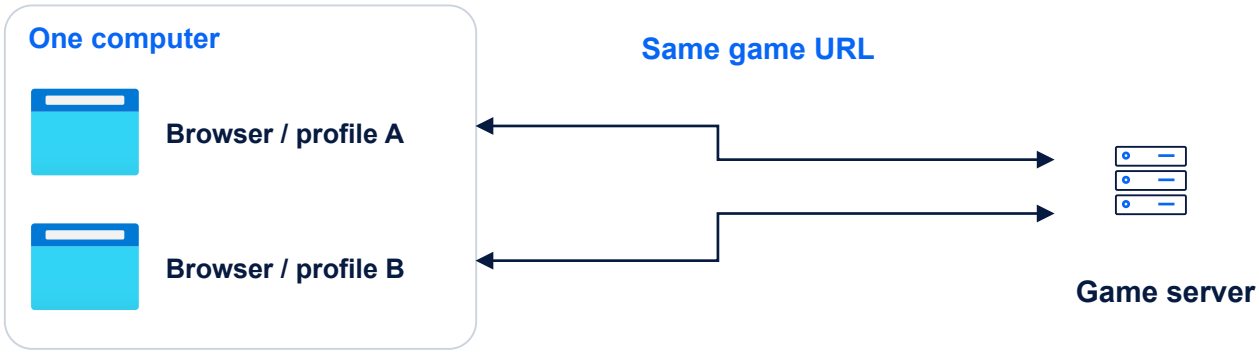


Figure 3: Browser sessions and shared game server.

Local multiplayer test procedure

- For the provided setup, start one Python game server and one Vite development server. Open the same URL in browser/profile A and browser/profile B; create or join the same room with two distinct players.
- Confirm that movement, collection, cloaking and match results are exchanged through that shared server. Do not start a separate game server for each player.
- Reload browser A and open a second tab in its profile. Verify that the stored identity is reused and that duplicate tabs follow the documented connection policy rather than creating another player.
- Verify that browser B retains a different identity, then test elimination and explicit rejoin with zero credits. Record the session setup and observed results.

Processing placement and networking libraries

Browser-side networking and socket libraries are allowed, alongside APIs such as `WebSocket`/`fetch` and Python networking libraries. Teams must design their own application protocol, room behavior and synchronization rather than delegate these to a ready-made multiplayer or replication platform (e.g., Colyseus, Nakama, Photon, Firebase Realtime Database or Supabase Realtime). UI and graphics libraries are also allowed.

Vite normally serves frontend HTML, JavaScript and CSS during development; the downloaded client executes in the browser, which may issue HTTP requests or open socket connections directly to the game server. Teams may instead choose server-side middleware, proxying or centralized processing, including deliberately extending the Vite setup. This placement is not prescribed. Explain the trade-off: centralization can simplify coordination and reduce client work, but may add network hops, server load and bottlenecks; browser-side processing can reduce those costs but increases client workload and requires

handling differing device capabilities. Keep authoritative state, credit rules and match outcomes under the game server's control.

Client–server communication

Figure 4 summarizes the communication roles. Teams must specify the messages, exchange sequences and transport used by their implementation.



Figure 4: Client–server communication.

5. Progressive implementation stages

All stages use the same codebase. Demonstrate correct game behavior first, then justify and measure networking improvements. Each stage produces evidence for the final submission. Figure 5 shows deployment components, not a required communication route. Vite provides the frontend during development; the browser runs the client, while Python runs the authoritative game server. Teams choose direct browser–server communication, middleware/proxy routing and the placement of non-authoritative processing, and must explain their trade-offs. Public hosting serves the built frontend and game endpoint, together or separately. Remote clients use distinct networks; Stage 4 compares baseline and improved runs under comparable conditions.

Stage requirements

0 Local Prototype
Environment: Localhost

Extract the starter ZIP downloaded from BK LMS. Set up the TypeScript client and Python game server. Serve the web client using the provided Vite setup or your chosen arrangement. Open the served game URL in a browser, build the minimal game screen and establish communication with the local server.

Acceptance: The browser can connect to the local server and display game state.

Evidence: Local demonstration and initial communication flow.

1 Protocol MVP
Environment: Localhost

Define communication requirements and implement the initial protocol and persistent player identity. Open the same game URL in two independent browser sessions on one computer. Preserve Move, Collect, Cloak, collision, round lifecycle and rejoin behavior.

Acceptance: Two independent browser sessions share one server, exchange actions and obtain correct game outcomes; reload and duplicate-tab tests preserve player identity.

Evidence: Protocol Table v1, at least one message-sequence diagram, functional results, one sample Browser Scenario Mode result and browser-session test evidence.

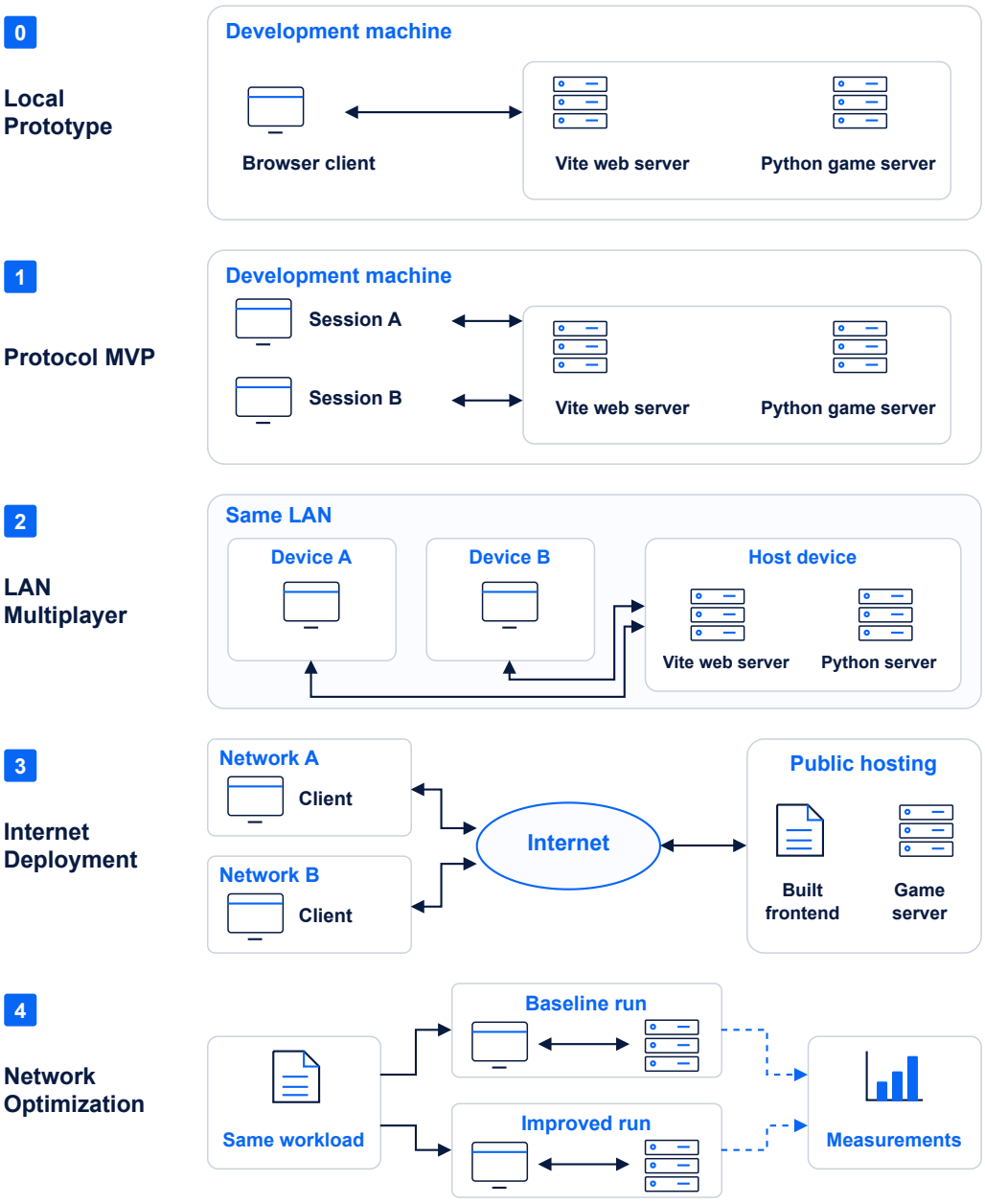


Figure 5: Deployment architecture across implementation stages.

2 LAN Multiplayer

Environment: LAN (2–4 physical devices)

Serve the game to separate physical devices on the same LAN. Verify joined rooms, player state, event delivery and behavior when a device disconnects and rejoins.

Acceptance: Game interactions work across hosts, with consistent completed-match results.

Evidence: LAN demonstration and test evidence.

3 Internet Deployment

Environment: Public Internet

Deploy the client and server on Render Free or an equivalent public endpoint. Verify remote gameplay from at least two distinct networks, run Browser Scenario Mode against the public deployment and record the baseline measurements.

Acceptance: A publicly accessible game supports remote multiplayer with correct outcomes.

Evidence: Public game URL and baseline results.

4 **Network Optimization**
Environment: Same deployment as the retained baseline

Identify a networking bottleneck or undesirable communication behavior, implement an improvement and repeat comparable tests. Compare lockstep and real-time scripted tests over your implemented game transport, and use simulated clients at increasing player counts.

Acceptance: The improvement is evaluated against a retained baseline, with game correctness preserved.

Evidence: Protocol Table v2, before/after measurements, charts and explanations.

Required final demonstration

Demonstrate real multiplayer play, explain your communication protocol and show experimental evidence. Every member should be able to explain the design choices and an implemented part of the system.

6. Implementation principles

- Keep the mechanics in Section 3 unchanged while redesigning networking, application state and delivery strategy.
- Teams select and justify their transport, message encoding and synchronization strategy rather than relying on ready-made multiplayer frameworks.
- Preserve baseline results and reproducible workloads so that proposed improvements can be evaluated fairly.
- Client synchronization and visualization quality may be improved, provided the authoritative game result remains correct.

7. Testing and experimental evaluation

At minimum, every team's application must preserve the fixed game mechanics when deployed over a network. Moving from localhost to LAN or the Internet must not change the required game behavior or completed-match outcomes under the same controlled scenario.

Teams can check this with Browser Scenario Mode in the starter ZIP: replay a predetermined multi-player scenario through the game's actual networking implementation and compare the resulting final game outcome with the published expected output. After submission, each application will also be checked in this way **before its networking optimizations are considered**. This common correctness check is why teams must not modify the game mechanics in Section 3.

Use fixed input/expected-output pairs from the starter ZIP, and design additional repeatable action timelines with `join`, `rejoin`, `leave` and `input` operations. Compare the winner, completed round, alive/eliminated roster and every player's remaining credits. Intermediate tick-by-tick states and final coordinates are not compared. Network measurements are recorded separately.

Figure 6 gives an **example input, expected output, observed output and comparison**. The notation and scenario in the figure are illustrative. Teams must design additional scenarios and write their own tables, narrative descriptions and explanations rather than copy this sample. For Browser Scenario Mode, retain the required input, expected-output and result JSON formats supplied in the starter ZIP so that the independent verifier can process them. Additional scenarios must use those formats; teams must not alter the supplied input/expected-output pairs. In this example, A contacts B before the first credit decay; A has more credits, and B's zero-credit balance yields no kill reward.

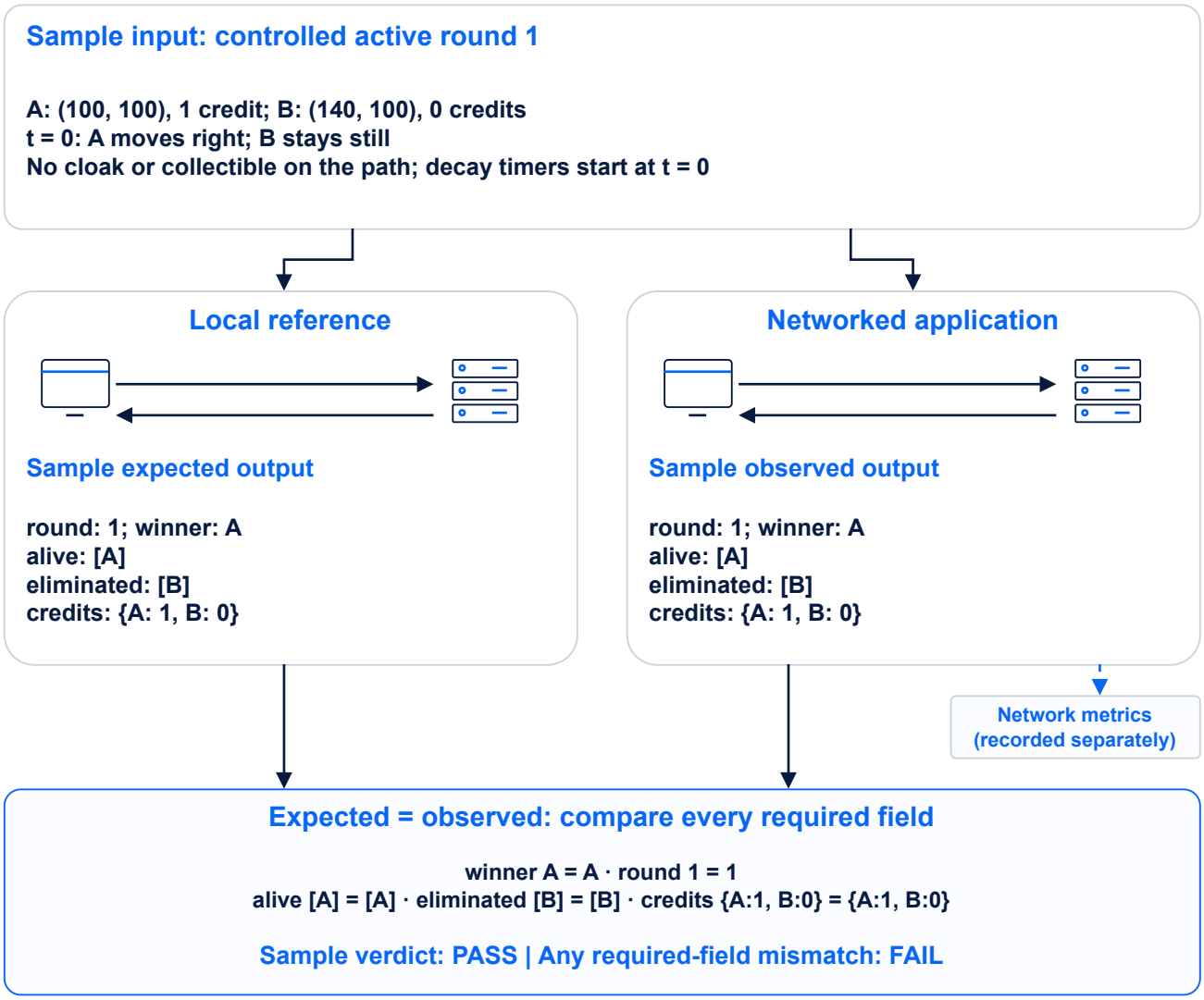


Figure 6: Scenario-based evaluation of completed-match outcomes.

Completed-match evaluation

Test property	Evaluation rule
Winner and completed round	Must match the expected scenario outcome.
Alive/eliminated participants	Must match the expected final roster.
Credits at the end of the match	Must match for each participating player.
Intermediate states or ticks	Need not match the oracle tick-by-tick.
Snapshot delivery and positions	May differ; no requirement for identical final coordinates.
Incomplete round	Used for functional diagnostics, not a completed-match verdict.

Delayed, duplicate or missing actions can alter the final game outcome. Your test methodology must detect and explain such differences. Apply controlled communication faults at the application layer or use tools such as Linux `tc netem` or a TCP proxy for network impairment. Distinguish intentional application-message drops from TCP/IP packet loss.

Network performance measurements

Metric	What to record	Purpose
Interaction latency	Defined application event latency; median and p95 if measurable.	Responsiveness.
Application RTT	Request/response probe timings and sampling method.	Path delay reference.
Bandwidth/traffic	Bytes/s, per-player traffic and protocol overhead.	Communication efficiency.
Concurrent clients	Gradually test 10, 25, 50 and up to 100 simulated players.	Load behavior.
Recovery	Disconnect/reconnect time and observed state convergence.	Robustness.
Correctness	Completed-match pass/fail under the same script.	Functional acceptance.

Scenario result verification and timing

Use the independent checker supplied in the ZIP:

```
python tools/verify_result.py --expected <scenario>.expected.json --result <run-id>.result.json
```

Keep the trusted expected file unchanged. The checker also verifies format identifiers, scenario name/hash, embedded expected values, the reported verdict and absence of simulator timeouts.

The reported tick-to-action-delivery p50/p95 measures elapsed time from a simulator tick request to the game server confirming receipt of the action, including browser scheduling and network delivery. It is neither TCP RTT nor full cross-player action-to-observation latency; measure those separately where reported.

> **Experimental comparison**
>
> Keep game mechanics, test workload and network conditions comparable before and after optimization. A 100-client run is a stress-test target; report measured limits and failures rather than hiding them. A networking optimization cannot gain credit by changing game rules or omitting required state/events.

Browser Scenario Mode: synchronized input/output testing

The starter ZIP includes **Browser Scenario Mode**, fixed `*.input.json` scenarios, matching `*.expected.json` results and a result verifier. A scenario specifies multiple players and actions at logical ticks. Each team's communication implementation must preserve the required **completed-match outcome**, not reproduce every intermediate snapshot.

> **Run the same scenario from multiple browsers**
>
> - Open the same game URL in two or more browser sessions or devices. Upload the **same** `*.input.json` in every browser.
> - Use one shared **Run ID** and assign a **different scripted player** to each browser. Every scripted player must have a participating browser.
> - Select the same test mode, connect and click **Ready** on each browser. The server starts automatically when all players are Ready; simultaneous manual Start clicks are unnecessary.
> - When the test finishes, download its `*.result.json`. Compare the observed final result against the fixed `*.expected.json` from the ZIP and preserve the evidence for your report.

Two complementary execution modes

Mode	Execution rule	Purpose
Correctness (lockstep)	The server requests scripted actions in a deterministic order within each logical tick, waits for confirmations from the **actual gameplay communication**, then advances the simulation.	Repeatable mechanics and protocol correctness.
Networking (real time)	All browsers share the Ready barrier, but the game runs at **20 ticks/s** without waiting for late player messages.	Evaluate effects of delay, stale input and transport design on responsiveness and outcomes.

> **Test the transport you actually implement**
>
> A separate *test control channel* coordinates the lobby, Ready barrier and scheduled actions. **Scripted gameplay messages must pass through the same communication mechanism used during normal play.** If your team changes the baseline WebSocket approach to HTTP polling, long polling or another transport, update the scenario data-plane adapter accordingly. A test that bypasses the gameplay transport does not establish networking correctness.

Fixed input, expected output and downloaded result

File	Role
`*.input.json`	Scripted player identities and `join`, `input`, `leave`, `rejoin` actions with logical tick positions.
`*.expected.json`	Fixed expected completed round, winner, final surviving-player roster and remaining credits.
`*.result.json`	Observed outcome, scenario identifier/hash, execution mode, reported comparison, action delivery latency, timing metrics and failures.

For deterministic correctness testing, a completed match is correct when its **observed final result matches the expected outcome**. In real-time mode, delayed actions can change the outcome; report and explain these differences separately rather than treating a lockstep PASS as evidence of low latency. Exact per-tick state, intermediate positions and snapshots **do not need to match**. Document action delays, missing input, timeouts and traffic separately. A submitted report's self-declared PASS is not sufficient: evaluation independently checks observed results against trusted expected files. Additional unseen scenarios may be used. The canonical outcome includes the finished phase, round, winner and alive/credit fields for player entries still in the room; players who have left are not required to remain in that roster.

8. Required submission

The handout and starter source ZIP are distributed through **BK LMS**. Submit the completed application source **as a ZIP through BK LMS**; no public source repository is required. Include run instructions and an accessible deployed game. Do not commit secrets or authentication credentials.

Deliverable	Required content
Source code	ZIP with runnable client/server source, dependency files, setup instructions and tests for independent browser sessions and persistent player identity.
Deployed application	Public multiplayer URL and concise joining instructions.
Protocol design	Team-authored message tables, data formats, interaction sequences, chosen transport and explanations of the implemented design; do not reproduce the handout samples directly.
Networking evaluation	Baseline vs. improvement, workload/scenario inputs, measurement method, graphs and analysis.
Correctness evidence	Synchronized browser scenario evidence: input JSON, trusted expected JSON, downloaded result JSON, final-outcome comparison and explanation of failures.
Demonstration and defense	Working demo, explanation of trade-offs and technical Q&A by team members.

9. Assessment rubric

Assessment area	Weight	Assessment focus
Requirements and protocol design	30%	Protocol semantics, clear specifications, sequence diagrams and justification.
Implementation and correctness	25%	Functional localhost, LAN and Internet gameplay, including final results.
Network optimization and experiments	25%	Hypotheses, comparable tests, latency/traffic/recovery measurements and reasoning.
Presentation and individual defense	20%	Working demonstration, technical clarity and each member's understanding.
Total	**100%**	

10. Getting started

- Review the fixed mechanics, particularly the timing and correctness requirements for moves, collections, collisions, invisibility and match completion.
- Download and extract the starter ZIP from BK LMS, then run the TypeScript client and Python server on localhost and inspect the provided scenario input/expected-output files.
- Run the sample scenario from separate browsers using the shared Ready barrier. Save the JSON result, document your protocol and compare both lockstep and real-time test behavior as you progress.

11. Contact and support

For questions or issues related to this assignment, contact your theory lecturer by email.

Author: MEng. Thanh Hoang Le Hai — thanhhoang@hcmut.edu.vn. You may also consult the lecturer responsible for your practical class.

12. Document version history

v1.2 — 24 October 2026

- Use Browser Scenario Mode for correctness evidence: load the same input file in all participating browsers, select distinct players and share one Run ID; the test starts when everyone is Ready (Section 7).
- Run lockstep tests for deterministic correctness and real-time tests for networking effects. Scripted actions must use your actual gameplay transport, including after you change that transport (Sections 5 and 7).
- Preserve the supplied JSON formats and input/expected-output pairs. Verify downloaded results with the independent checker, include the evidence in your submission and prepare for additional unseen scenarios (Sections 7 and 8).
- Download the starter ZIP and submit your source ZIP through BK LMS. No public source repository is required (Section 8).
- Game mechanics and assessment weights are unchanged.

v1.1 — 22 October 2026

- Test independent browser sessions against one shared server; implement persistent player identity across reloads and same-profile tabs, with a defined duplicate-tab policy (Section 4).
- Choose a distinctive theme without changing mechanics. Write your own protocol tables and explanations; supplied examples are illustrative (Sections 2 and 4).
- Browser socket libraries and different communication/processing arrangements are permitted. Explain the trade-offs of your chosen design (Section 4).
- Preserve game outcomes over the network: scenario-based correctness checks precede consideration of optimizations (Section 7).
- Keep source code and solutions private during the course. After the course, you may publish your own work, excluding restricted supplied materials without permission (page 1).
- Game mechanics and assessment weights are unchanged from v1.0.

v1.0 — Initial version

- Initial assignment requirements, game mechanics, implementation stages and assessment rubric.